**1. Implement a singly linked list with the following operations:**
**- Insert at the beginning, end and a given position.**
**- Delete from the beginning, end and a given position.**
**- Display the list.**

**Answer**:

```
##include <stdio.h>
#include <stdlib.h>
struct Node {
    char data;
    struct Node* next;};
int main() {
    struct Node *head = NULL, *temp = NULL;
    struct Node *a, *a2, *a10, *a4, *a100, *a1, *a3, *a5;
    a = (struct Node*)malloc(sizeof(struct Node));
    a->data = 'r';
    a2 = (struct Node*)malloc(sizeof(struct Node));
    a2->data = 'a';
    a10 = (struct Node*)malloc(sizeof(struct Node));
    a10->data = 'k';
    a4 = (struct Node*)malloc(sizeof(struct Node));
    a4->data = 'i';
    a100 = (struct Node*)malloc(sizeof(struct Node));
    a100->data = 'b';
    a100->next = NULL;
    a->next = a2;
    a2->next = a10;
    a10->next = a4;
    a4->next = a100;
    head = a;
    printf("Initial list:\n");
    temp = head;
    while (temp != NULL) {
       printf("%c -> ", temp->data);
       temp = temp->next;}
    printf("NULL\n");
    printf("\nDelete from start.\n");
       head = head->next;
```

```
    free(temp);
temp = head;
printf("List after deleting from start: ");
while (temp != NULL) {
    printf("%c -> ", temp->data);
    temp = temp->next;}
printf("NULL\n");
printf("\nDelete from end.\n");
a4->next = NULL;
    free(a100);
temp = head;
printf("List after deleting from end: ");
while (temp != NULL) {
    printf("%c -> ", temp->data);
    temp = temp->next;}
printf("NULL\n");
printf("\nDelete from position .\n" );
a2->next = a4;
temp=head;
printf("List after deleting from position: ");
while (temp != NULL) {
    printf("%c -> ", temp->data);
    temp = temp->next;}
printf("NULL\n");
printf("\nInsertion at start.\n");
a1 = (struct Node*)malloc(sizeof(struct Node));
a1->data = 'F';
a1->next = head;
head = a1;
temp = head;
printf("List after inserting at start: ");
while (temp != NULL) {
    printf("%c -> ", temp->data);
    temp = temp->next;}
printf("NULL\n");
printf("\nInsertion at end.\n");
a5 = (struct Node*)malloc(sizeof(struct Node));
```

```
    a5->data = 'm';
    a5->next = NULL;
    a4->next = a5;
    temp = head;
    printf("List after inserting at end: ");
    while (temp != NULL) {
        printf("%c -> ", temp->data);
        temp = temp->next;}
    printf("NULL\n");
    printf("\nInsertion at position .\n");
    a3 = (struct Node*)malloc(sizeof(struct Node));
    a3->data = 'h';
    a3->next = a4;
    a2->next = a3;
    temp = head;
    printf("List after inserting at position: ");
    while (temp != NULL) {
        printf("%c -> ", temp->data);
        temp = temp->next;}
    printf("NULL\n");
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);}
    return 0;}
```

**Output:**

```
Initial list:
r -> a -> k -> i -> b -> NULL

Delete from start.
List after deleting from start: a -> k -> i -> b -> NULL

Delete from end.
List after deleting from end: a -> k -> i -> NULL

Delete from position .
List after deleting from position: a -> i -> NULL
```

Insertion at start.
List after inserting at start: F -> a -> i -> NULL

Insertion at end.
List after inserting at end: F -> a -> i -> m -> NULL

Insertion at position .
List after inserting at position: F -> a -> h -> i -> m -> NULL

**2. WImplement a stack using an array with the following operations:**
**- Push, Pop, Peek, Display.**
**Answer:**

```
#include <stdio.h>
#define SIZE 10
int main() {
    int stack[SIZE];
    int top = -1,i;
    printf("Push 3 elements: 10, 20, 30\n");
    if (top>=SIZE-1){printf("Overflow, Stack is full.\n");}
    else {
        top++;
        stack[top] = 10;
        top++;
        stack[top] = 20;
        top++;
        stack[top] = 30;}
    printf("Stack after pushing 3 elements:\n");
    for (i = top; i >= 0; i--) {
        printf("%d\n", stack[i]);}

    if (top==-1){printf("Underflow, Stack is empty.\n");}
    else{printf("Top element (peek): %d\n", stack[top]);}
    printf("Pop top element\n");
    if (top==-1){printf("Underflow, Stack is empty.\n");}
    else {
```

```
        int popped = stack[top];
        top--;
        printf("Popped element: %d\n", popped);}
    printf("Stack after popping one element:\n");
    for (i = top; i >= 0; i--) {
        printf("%d\n", stack[i]);}
    printf("Push 40\n");
    if (top>=SIZE-1){printf("Overflow, Stack is full.\n");}
    else {
        top++;
        stack[top] = 40;}
    printf("Final stack elements:\n");
    for (i = top; i >= 0; i--) {
        printf("%d\n", stack[i]);}
    return 0;}
```

**Output:**

```
Push 3 elements: 10, 20, 30
Stack after pushing 3 elements:
30
20
10
Top element (peek): 30
Pop top element
Popped element: 30
Stack after popping one element:
20
10
Push 40
Final stack elements:
40
20
10
```

**3. Implement a queue using an array with the following operations:**
**- Enqueue, Dequeue, Peek, Display.**

**Answer**:

```
#include <stdio.h>
```

```
#define SIZE 10
int main() {
    int queue[SIZE];
    int front = -1, rear = -1,i;
    printf("Enqueue 3 elements: 10, 20, 30\n");
    if (rear == SIZE - 1) {
        printf("Overflow, Queue is full.\n");
    } else {
        if (front == -1) front = 0;
        rear++;
        queue[rear] = 10;
        rear++;
        queue[rear] = 20;
        rear++;
        queue[rear] = 30;}
    if (front == -1) {
        printf("Underflow, Queue is empty.\n");
    } else {
        printf("Queue after enqueueing 3 elements:\n");
        for (i = front; i <= rear; i++) {
            printf("%d ", queue[i]);}
        printf("\n");}
    if (front == -1) {
        printf("Underflow, Queue is empty.\n");
    } else {
        printf("Peek element: %d\n", queue[front]);}
    printf("Dequeue an element\n");
    if (front == -1 || front > rear) {
        printf("Underflow, Queue is empty.\n");
    } else {
        int dequeued = queue[front];
        front++;
        printf("Dequeued element: %d\n", dequeued);
        if (front > rear) {
            front = rear = -1;}}
    if (front == -1) {
        printf("Underflow, Queue is empty.\n");
```

```
    } else {
        printf("Queue after dequeue operation:\n");
        for (i = front; i <= rear; i++) {
            printf("%d ", queue[i]);}
        printf("\n");}
    printf("Enqueue 40\n");
    if (rear == SIZE - 1) {
        printf("Overflow, Queue is full.\n");
    } else {
        if (front == -1) front = 0;
        rear++;
        queue[rear] = 40;}
    if (front == -1) {
        printf("Queue is empty.\n");
    } else {
        printf("Final queue elements:\n");
        for (i = front; i <= rear; i++) {
            printf("%d ", queue[i]);}
        printf("\n");}
    return 0;}
```

**Output:**

```
Enqueue 3 elements: 10, 20, 30
Queue after enqueueing 3 elements:
10 20 30
Peek element: 10
Dequeue an element
Dequeued element: 10
Queue after dequeue operation:
20 30
Enqueue 40
Final queue elements:
20 30 40
```

**4.Implement a binary tree and insert nodes into the binary tree recursively with the following traversals:**
**- In-order, Pre-order, Post-order.**

**Answer**:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;};
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;}
struct Node* insert(struct Node* root) {
    int v=0;
    printf("Enter data(-1 for NULL): ");
    scanf("%d",&v);
    if (v == -1) {
        return NULL;}
    struct Node* newNode = createNode(v);
    printf("left of %d(-1 for NULL): ",v);
        newNode->left = insert(newNode->left);
    printf("right of %d(-1 for NULL): ",v);
        newNode->right = insert(newNode->right);
    return newNode;}
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);}}
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);}}
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
```

```
        postorder(root->right);
        printf("%d ", root->data);}}
int main() {
    struct Node* root = NULL;
    root = insert(root);
    printf("In-order traversal: ");
    inorder(root);
    printf("\n");
    printf("Pre-order traversal: ");
    preorder(root);
    printf("\n");
    printf("Post-order traversal: ");
    postorder(root);
    printf("\n");
    return 0;}
```

**Output**:

```
Enter data(-1 for NULL): 50
left of 50(-1 for NULL): Enter data(-1 for NULL): 10
left of 10(-1 for NULL): Enter data(-1 for NULL): 20
left of 20(-1 for NULL): Enter data(-1 for NULL): 30
left of 30(-1 for NULL): Enter data(-1 for NULL): -1
right of 30(-1 for NULL): Enter data(-1 for NULL): -1
right of 20(-1 for NULL): Enter data(-1 for NULL): -1
right of 10(-1 for NULL): Enter data(-1 for NULL): 25
left of 25(-1 for NULL): Enter data(-1 for NULL): -1
right of 25(-1 for NULL): Enter data(-1 for NULL): -1
right of 50(-1 for NULL): Enter data(-1 for NULL): 60
left of 60(-1 for NULL): Enter data(-1 for NULL): -1
right of 60(-1 for NULL): Enter data(-1 for NULL): 8
left of 8(-1 for NULL): Enter data(-1 for NULL): -1
right of 8(-1 for NULL): Enter data(-1 for NULL): 5
left of 5(-1 for NULL): Enter data(-1 for NULL): -1
right of 5(-1 for NULL): Enter data(-1 for NULL): -1
In-order traversal: 30 20 10 25 50 60 8 5
Pre-order traversal: 50 10 20 30 25 60 8 5
Post-order traversal: 30 20 25 10 5 8 60 50
```

**5.Count the total number of nodes and leaf nodes in a binary tree.**

**Answer:**

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;};
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;}
struct Node* insert(struct Node* root) {
    int v=0;
    printf("Enter data(-1 for NULL): ");
    scanf("%d",&v);
    if (v == -1) {
        return NULL;}
    struct Node* newNode = createNode(v);
    printf("left of %d(-1 for NULL): ",v);
        newNode->left = insert(newNode->left);
    printf("right of %d(-1 for NULL): ",v);
        newNode->right = insert(newNode->right);
    return newNode;}
    int countNodes(struct Node* root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);}
int countLeafNodes(struct Node* root) {
    if (root == NULL) return 0;
    if (root->left == NULL && root->right == NULL) return 1;
    return countLeafNodes(root->left) + countLeafNodes(root->right);}
int main() {
    struct Node* root = NULL;
    root = insert(root);
    printf("\nTotal nodes: %d\n", countNodes(root));
```

```
    printf("Leaf nodes: %d\n", countLeafNodes(root));
    return 0;}
```

**Output:**

```
Enter data(-1 for NULL): 50
left of 50(-1 for NULL): Enter data(-1 for NULL): 10
left of 10(-1 for NULL): Enter data(-1 for NULL): 20
left of 20(-1 for NULL): Enter data(-1 for NULL): 30
left of 30(-1 for NULL): Enter data(-1 for NULL): -1
right of 30(-1 for NULL): Enter data(-1 for NULL): -1
right of 20(-1 for NULL): Enter data(-1 for NULL): -1
right of 10(-1 for NULL): Enter data(-1 for NULL): 25
left of 25(-1 for NULL): Enter data(-1 for NULL): -1
right of 25(-1 for NULL): Enter data(-1 for NULL): -1
right of 50(-1 for NULL): Enter data(-1 for NULL): 60
left of 60(-1 for NULL): Enter data(-1 for NULL): -1
right of 60(-1 for NULL): Enter data(-1 for NULL): 8
left of 8(-1 for NULL): Enter data(-1 for NULL): -1
right of 8(-1 for NULL): Enter data(-1 for NULL): 5
left of 5(-1 for NULL): Enter data(-1 for NULL): -1
right of 5(-1 for NULL): Enter data(-1 for NULL): -1

Total nodes: 8
Leaf nodes: 3
```

**6.Implement a Binary Search Tree (BST) with In-order Traversal.**

**Answer:**

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;};
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
```

```
    return newNode;}
struct Node* insert(struct Node* root, int data) {
    if (root == NULL) {
        return createNode(data);}
    if (data < root->data) {
        root->left = insert(root->left, data);
    } else {
        root->right = insert(root->right, data);}
    return root;}
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);}}
int main() {
    struct Node* root = NULL;
    int values[] = {30,20,10,25,50,60,8,5};
    int n = sizeof(values) / sizeof(values[0]);
    for (int i = 0; i < n; i++) {
        root = insert(root, values[i]);}
    printf("In-order traversal: ");
    inorder(root);
    printf("\n");
    return 0;}
```

**Output**:

In-order traversal: 5 8 10 20 25 30 50 60

**7.Write recursive functions for the following:**

**- Factorial of a number, Fibonacci series, Binary search, Tower of Hanoi.**

**Answer:**

```
#include <stdio.h>
int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    return n * factorial(n - 1);}
int fibonacci(int n) {
    if (n == 0)
```

```
        return 0;
    if (n == 1)
        return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);}
int binarySearch(int arr[], int low, int high, int key) {
    if (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (key < arr[mid])
            return binarySearch(arr, low, mid - 1, key);
        else
            return binarySearch(arr, mid + 1, high, key);}
    return -1;}
void towerOfHanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", from, to);
        return;}
    towerOfHanoi(n - 1, from, aux, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    towerOfHanoi(n - 1, aux, to, from);}
int main() {
    int num = 5;
    printf("Factorial of %d = %d\n", num, factorial(num));
    printf("Fibonacci series (first %d terms): ", num);
    for (int i = 0; i < num; i++) {
        printf("%d ", fibonacci(i));}
    printf("\n");
    int arr[] = {5,8,10,20,25,30,50,60};
    int size = sizeof(arr) / sizeof(arr[0]);
    int key = 25;
    int index = binarySearch(arr, 0, size - 1, key);
    if (index != -1)
        printf("Element %d found at index %d\n", key, index);
    else
        printf("Element %d not found\n", key);
    int disks = 3;
```

```
    printf("Tower of Hanoi steps for %d disks:\n", disks);
    towerOfHanoi(disks, 'A', 'C', 'B');
    return 0;}
```

**Output:**

```
Factorial of 5 = 120
Fibonacci series (first 5 terms): 0 1 1 2 3
Element 25 found at index 4
Tower of Hanoi steps for 3 disks:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```